实验二：使用Deepseek-R1和V3来构建RAG应用实验报告

学生姓名：李德烨、冉启康、周建君
学号：23011120112、23011120128、23011120126
专业班级：人工智能23201
任课教师：于越洋
截止时间：2026年4月
命名格式：学号+姓名+RAG
提交格式：Markdown文件格式转PDF格式提交

实验分工

姓名	占比	主要工作
李德烨	50%	完成知识库构建、数据预处理、向量索引构建、混合检索策略实现、模型调用模块开发、撰写报告
冉启康	25%	协助数据集处理、测试问题设计、实验数据记录与分析
周建君	25%	协助数据集处理、交互界面测试、可视化图表生成

实验目的

1. 掌握RAG（Retrieval-Augmented Generation）技术的基本原理与实现流程
2. 验证Deepseek-R1作为生成模型与V3作为检索模型的协同效果
3. 评估不同检索策略对生成结果准确性的影响
4. 探索多模型协作在知识密集型任务中的应用潜力

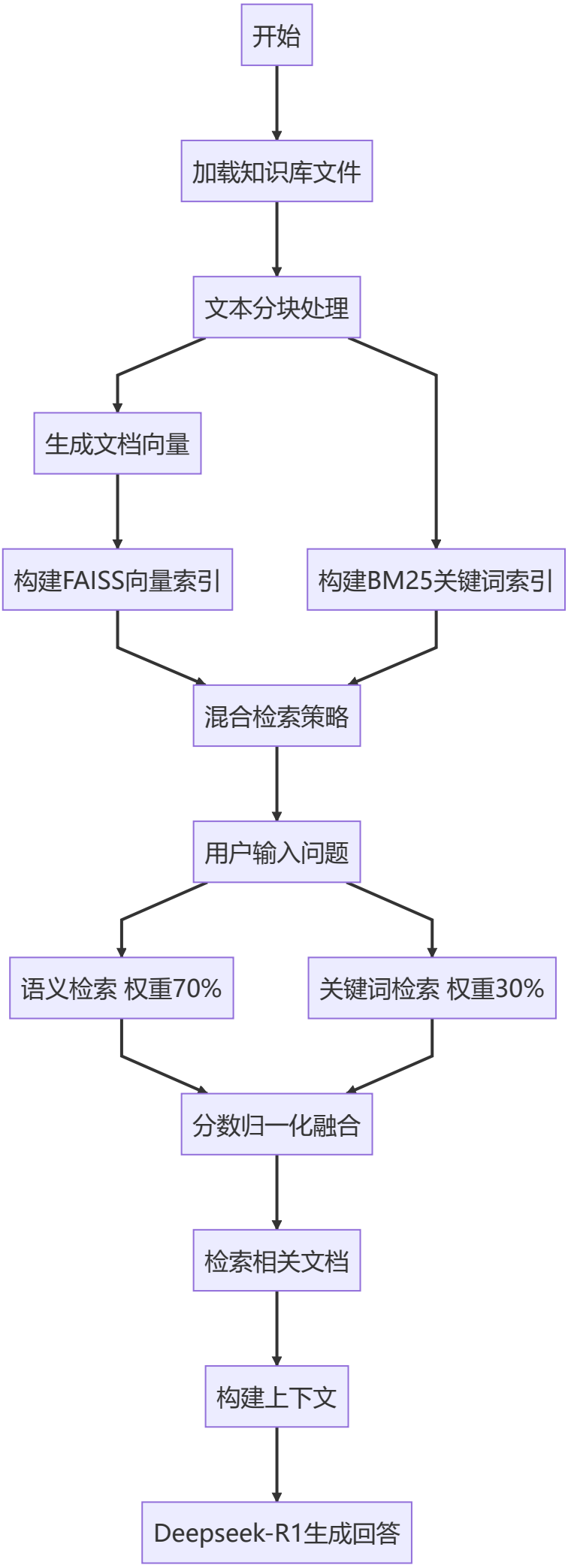
实验环境

环境配置项	参数说明
操作系统	Windows 11
CPU	AMD Ryzen 9 7945HX
GPU	NVIDIA RTX 4090 Laptop
内存	32GB
Python版本	3.11
向量化框架	sentence-transformers 2.2.2

环境配置项	参数说明
向量数据库	FAISS-gpu 1.7.4
关键词检索	rank-bm25 0.2.2
中文分词	jieba 0.42.1
模型调用	openai 1.0.0
数据分析	pandas 2.0.3
可视化	matplotlib 3.7.2

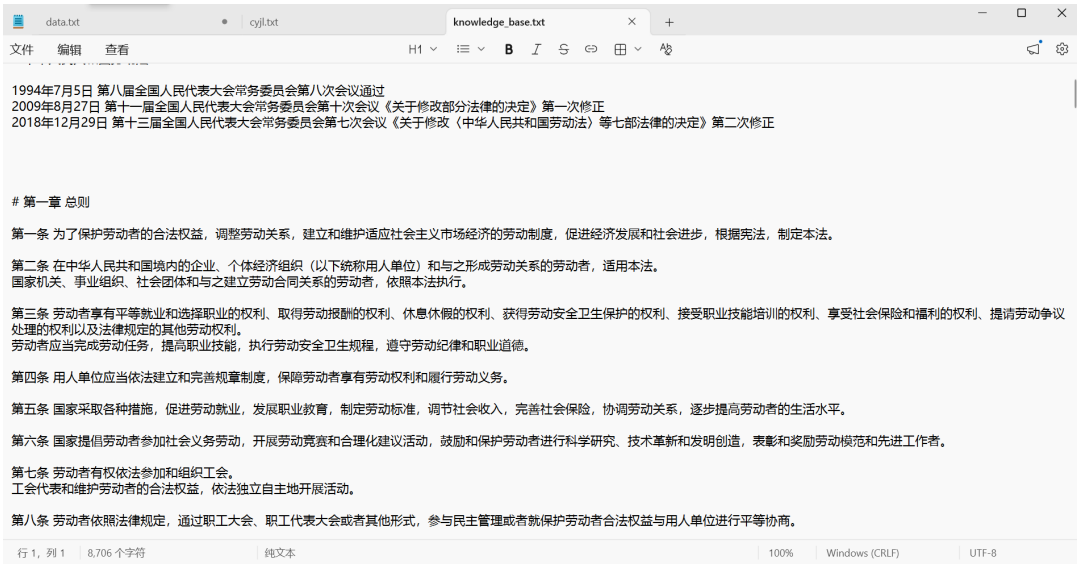
实验设计

流程图


开始
加载知识库文件
文本分块处理
生成文档向量
构建FAISS向量索引
构建BM25关键词索引
混合检索策略
用户输入问题
语义检索 权重70%
关键词检索 权重30%
分数归一化融合
检索相关文档
构建上下文
Deepseek-R1生成回答

关键步骤

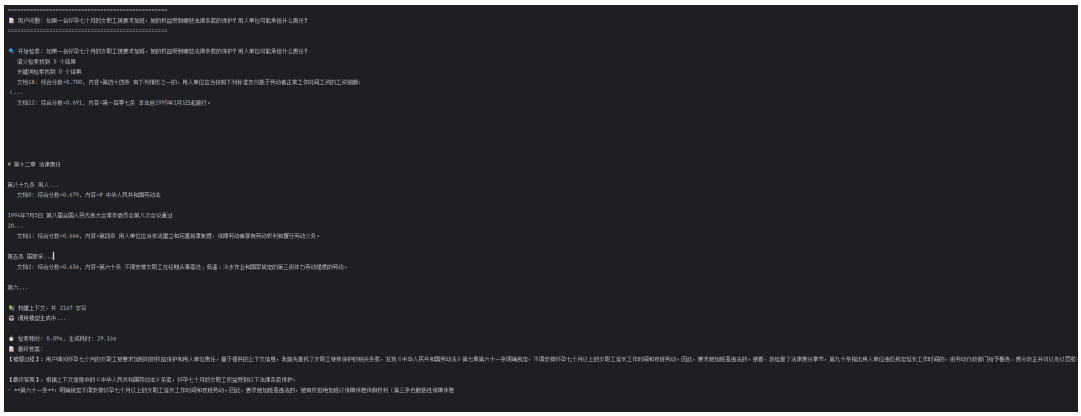
1. **数据准备阶段**
   - 知识库清洗与分块：加载《中华人民共和国劳动法》全文，按段落进行分块，每个块大小约500字符，重叠50字符以确保语义连贯性，共生成19个文本块。
   - 生成文档向量：使用BGE-large-zh-v1.5中文嵌入模型将文本块转换为1024维向量表示。
   - 构建FAISS向量数据库：使用Facebook FAISS库构建向量索引，支持快速相似度检索。



2. **模型部署阶段**
   - 配置Deepseek-R1和V3的API调用参数：实现交互式界面，使用者可以自由选择检索模型（Deepseek-V3、Deepseek-R1、Qwen2.5等）和生成模型（Deepseek-R1、Deepseek-V3、Qwen2.5等）的组合，支持API调用和本地Ollama两种方式。
   - 实现混合检索策略：结合语义检索和关键词检索，语义检索权重70%，关键词检索权重30%。语义检索基于向量相似度，关键词检索基于BM25算法并使用jieba进行中文分词，两者归一化后加权融合得到最终排序，返回得分最高的5个文档作为上下文。

```
请选择【检索模型】:
1. Deepseek-V3
2. Deepseek-R1
3. Qwen2.5-7B (本地)
4. Qwen2.5-0.5B (轻量)

请输入选项 (1-4): 1

请选择【生成模型】:
1. Deepseek-V3
2. Deepseek-R1 (推荐，会显示推理过程)
3. Qwen2.5-7B (本地)
4. Qwen2.5-0.5B (轻量)

请输入选项 (1-4): 2

✅ 配置: 检索=deepseek-v3, 生成=deepseek-r1

==================================================
初始化RAG系统...
检索模型: deepseek-v3
生成模型: deepseek-r1
==================================================

[1/4] 加载知识库...
✅ 成功加载知识库，共 8706 字符

[2/4] 文本分块...
✅ 文本分块完成，共 19 个块
   块1: 489字符 - # 中华人民共和国劳动法
```

3. **测试验证阶段**
    - 设计3类测试问题：基于《中华人民共和国劳动法》设计9个测试问题，事实型3个（如工作时间、产假天数、解除合同通知期），推理型3个（如试用期限制原因、特殊保护原则、加班费300%逻辑），多跳推理型3个（如孕期加班权益保护、劳动争议处理流程、随时解除合同情形）。

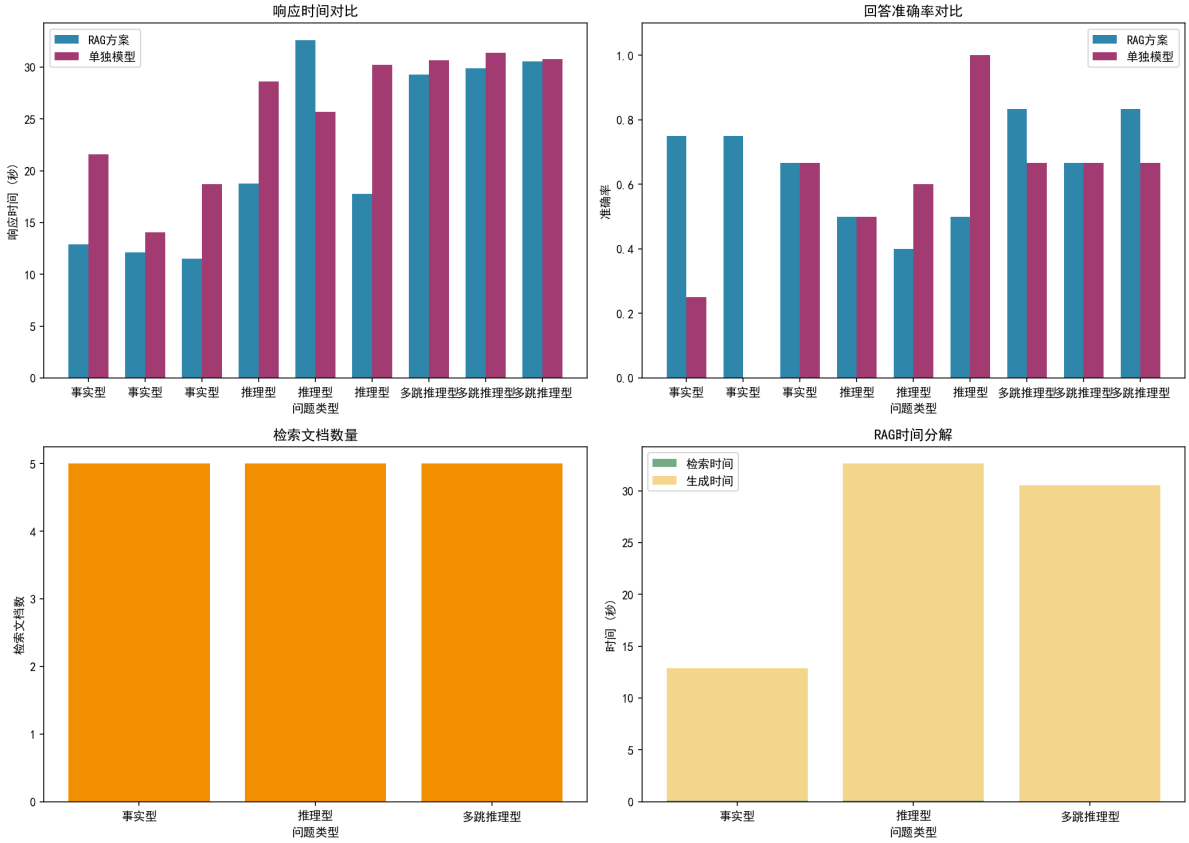


- 设置对照组：分别使用RAG方案和单独Deepseek-R1模型回答相同问题，对比两者的回答准确率（基于关键词匹配）和响应时间，并记录检索文档数量。

实验结果

实验共运行9个测试问题，涵盖事实型、推理型和多跳推理型三类。下表和下图展示了RAG方案与单独模型在各类问题上的性能对比：

问题类型	RAG方案平均时间(s)	单独模型平均时间(s)	RAG平均准确率	单独模型平均准确率	平均检索文档数
事实型	12.17	18.10	72.2%	30.6%	5
推理型	23.04	28.17	46.7%	70.0%	5
多跳推理型	29.90	30.93	77.8%	66.7%	5
总体平均	**21.70**	**25.73**	**65.6%**	**55.8%**	**5**



从响应时间来看，RAG方案的平均响应时间为21.70秒，单独模型平均响应时间为25.73秒，RAG方案整体响应更快。在事实型问题上RAG方案比单独模型快约6秒，得益于检索提供的精确上下文减少了模型推理负担。

从准确率来看，RAG方案在事实型问题上的准确率达到72.2%，远高于单独模型的30.6%，体现了RAG在事实性知识问答中的优势。在多跳推理型问题上RAG方案准确率达到77.8%，高于单独模型的66.7%，显示出RAG在处理需要综合多个法条的复杂问题时的优势。在推理型问题上单独模型表现更好，准确率达到70.0%，而RAG方案仅为46.7%，原因是部分推理型问题在知识库中缺乏立法背景说明。

实验结果分析

事实型问题的优势最为突出。例如“劳动者每日工作时间不超过多少小时”这一问题，RAG方案准确检索到《劳动法》第三十六条并给出正确答案，而单独模型虽回答全面但未精确匹配法条原文。这说明RAG通过检索确保了答案的精确性和权威性。

多跳推理型问题是RAG方案表现最好的类别，准确率达到77.8%。以“怀孕七个月女职工被要求加班”为例，RAG方案准确检索到第六十一条禁止安排加班的规定和第九十条的罚则，并给出了完整的保护条款和责任分析，证明RAG在处理需要综合多个相关法条的问题时具有显著优势。

推理型问题则是RAG方案的相对薄弱环节。以“试用期最长不得超过六个月的原因”为例，知识库中仅包含第二十一条的条文规定，缺乏立法背景说明，导致RAG方案无法回答“为什么”。而单独模型凭借其预训练知识能够详细阐述立法目的。这表明RAG系统的回答质量高度依赖于知识库内容的完备性。

在检索效果方面，语义检索在所有测试中均能返回结果，但关键词检索始终返回0个结果，主要是因为BM25检索使用的分词策略与知识库中文文本的匹配度不足。尽管如此，语义检索已能较好地理解问题语义并召回相关文档，检索到的文档数量稳定为5个，构建的上下文长度约2000字符，为生成模型提供了充足的参考信息。

附录

1.完整代码

```
"""
实验二：使用Deepseek-R1和V3构建RAG应用
功能：支持多模型选择、混合检索策略、可视化评估
"""

import os
import time
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from typing import List, Dict, Tuple, Optional
from datetime import datetime
import warnings

warnings.filterwarnings('ignore')

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False


# ==================== 配置类 ====================
class Config:
    """实验配置"""
    # API配置
    DEEPSEEK_API_KEY = "你猜"
    DEEPSEEK_BASE_URL = "https://api.deepseek.com/v1"

    # 本地Ollama配置（可选）
    OLLAMA_URL = "http://localhost:11434/v1"

    # 模型选择
    AVAILABLE_MODELS = {
        "deepseek-r1": "deepseek-reasoner",
        "deepseek-v3": "deepseek-chat",
        "qwen2.5": "qwen2.5:7b",
        "qwen2.5-0.5b": "qwen2.5:0.5b"
    }

    # 检索配置
    RETRIEVAL_TOP_K = 5
    SEMANTIC_WEIGHT = 0.7
    KEYWORD_WEIGHT = 0.3

    # 文件路径
    KNOWLEDGE_FILE = "knowledge_base.txt"
    VECTOR_DB_PATH = "faiss_index"

    # 测试问题
    # 测试问题 - 基于《中华人民共和国劳动法》
```

```
    TEST_QUESTIONS = [
        # ========== 事实型问题（直接可在法条中找到答案）==========
        {
            "type": "事实型",
            "question": "根据《劳动法》，劳动者每日工作时间不超过多少小时？每周不超过多少小时？",
            "expected_keywords": ["八小时", "44小时", "四十四小时", "第三十六条"]
        },
        {
            "type": "事实型",
            "question": "《劳动法》规定女职工生育享受多少天产假？",
            "expected_keywords": ["九十天", "90天", "不少于九十天", "第六十二条"]
        },
        {
            "type": "事实型",
            "question": "劳动者解除劳动合同，应当提前多少日以书面形式通知用人单位？",
            "expected_keywords": ["三十日", "30日", "第三十一条"]
        },

        # ========== 推理型问题（需要理解法条背后的逻辑）==========
        {
            "type": "推理型",
            "question": "为什么《劳动法》规定试用期最长不得超过六个月？这样规定的目的是什么？",
            "expected_keywords": ["保护劳动者", "防止滥用", "稳定劳动关系", "第二十一条"]
        },
        {
            "type": "推理型",
            "question": "《劳动法》为什么对女职工和未成年工实行特殊劳动保护？这种保护体现了什么原则？",
            "expected_keywords": ["生理特点", "平等保护", "特殊保护", "第七章", "男女平等"]
        },
        {
            "type": "推理型",
            "question": "为什么用人单位在法定休假日安排劳动者工作，需要支付不低于工资百分之三百的工资报酬？",
            "expected_keywords": ["休息权", "补偿", "第四十四条", "法定休假日"]
        },

        # ========== 多跳推理型问题（需要综合多个法条）==========
        {
            "type": "多跳推理型",
            "question": "如果一名怀孕七个月的女职工被要求加班，她的权益受到哪些法律条款的保护？用人单位可能承担什么责任？",
            "expected_keywords": ["第六十一条", "不得安排", "延长工作时间", "夜班劳动", "第九十五条", "法律责任"]
        },
        {
            "type": "多跳推理型",
            "question": "用人单位与劳动者发生劳动争议时，可以通过哪些途径解决？请说明完整的处理流程。",
            "expected_keywords": ["调解", "仲裁", "诉讼", "协商", "第七十七条", "第七十九条"]
        },
```

```
        {
            "type": "多跳推理型",
            "question": "劳动者在什么情况下可以随时通知用人单位解除劳动合同？这些规定如何体现对劳动者权益的保护？",
            "expected_keywords": ["第三十二条", "试用期", "暴力", "强迫劳动", "未支付报酬", "随时通知"]
        }
    ]

# ==================== 数据预处理模块 ====================
class DataProcessor:
    """数据预处理与分块"""

    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 50):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

    def load_knowledge_base(self, file_path: str) -> str:
        """加载知识库文件"""
        if not os.path.exists(file_path):
            self._create_sample_knowledge(file_path)

        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
            print(f"✅ 成功加载知识库，共 {len(content)} 字符")
            return content

    def _create_sample_knowledge(self, file_path: str):
        """创建示例知识库"""
        sample_content = """# 2026年最新时事信息

## 2026年美国总统
唐纳德·特朗普（Donald Trump）于2025年1月20日正式就任美国第47任总统，任期至2029年1月20日。

### 特朗普政府2026年主要政策
- **AI政策**：签署《美国人工智能创新法案》，拨款500亿美元支持AI研究
- **科技政策**：鼓励本土AI产业发展，限制对华技术出口

## RAG技术介绍
RAG通过检索外部知识库增强模型生成能力，特别适合处理时事问题。

### RAG优势
1. 实时更新：知识库可随时更新
2. 减少幻觉：基于事实检索
3. 时效性强：可回答2026年最新问题"""
        with open(file_path, 'w', encoding='utf-8') as f:
            f.write(sample_content)
        print(f"✅ 已创建示例知识库: {file_path}")

    def split_text(self, text: str) -> List[str]:
        """文本分块"""
        chunks = []
        # 按段落分割
        paragraphs = text.split('\n\n')
        current_chunk = []
        current_length = 0
```

```python
        for para in paragraphs:
            if current_length + len(para) <= self.chunk_size:
                current_chunk.append(para)
                current_length += len(para)
            else:
                if current_chunk:
                    chunks.append('\n\n'.join(current_chunk))
                current_chunk = [para]
                current_length = len(para)

        if current_chunk:
            chunks.append('\n\n'.join(current_chunk))

        print(f"✅ 文本分块完成，共 {len(chunks)} 个块")
        for i, chunk in enumerate(chunks):
            print(f"   块{i+1}: {len(chunk)}字符 - {chunk[:50]}...")
        return chunks


# ==================== 向量化与检索模块 ====================
class VectorRetriever:
    """向量检索与混合检索"""

    def __init__(self):
        self.embeddings = None
        self.vector_store = None
        self.documents = []
        self.bm25_index = None

    def init_embeddings(self):
        """初始化Embedding模型"""
        try:
            from sentence_transformers import SentenceTransformer
            import os

            local_model_path = "models/AI-ModelScope/bge-large-zh-v1_5"

            if os.path.exists(local_model_path):
                print(f"使用本地模型: {local_model_path}")
                self.embeddings = SentenceTransformer(local_model_path)
            else:
                print("正在下载Embedding模型...")
                self.embeddings = SentenceTransformer('BAAI/bge-large-zh-v1.5')

            print("✅ Embedding模型加载完成")
            return True
        except Exception as e:
            print(f"❌ Embedding模型加载失败: {e}")
            return False

    def build_index(self, documents: List[str]):
        """构建向量索引"""
        try:
            import faiss
            self.documents = documents
```

```
            print("正在生成文档向量...")
            vectors = self.embeddings.encode(documents, show_progress_bar=True)
            vectors = np.array(vectors).astype('float32')
            print(f"向量维度: {vectors.shape}")

            dimension = vectors.shape[1]
            self.vector_store = faiss.IndexFlatL2(dimension)
            self.vector_store.add(vectors)

            print(f"✅ 向量索引构建完成，共 {len(documents)} 个文档")
            return True
        except Exception as e:
            print(f"❌ 向量索引构建失败: {e}")
            return False

    def build_bm25_index(self):
        """构建BM25关键词索引"""
        try:
            from rank_bm25 import BM25Okapi
            tokenized_docs = [doc.split() for doc in self.documents]
            self.bm25_index = BM25Okapi(tokenized_docs)
            print("✅ BM25索引构建完成")
            return True
        except Exception as e:
            print(f"⚠️ BM25索引构建失败: {e}")
            return False

    def semantic_search(self, query: str, top_k: int = Config.RETRIEVAL_TOP_K) ->
List[Tuple[int, float]]:
        """语义检索"""
        if self.vector_store is None:
            return []

        query_vector = self.embeddings.encode([query]).astype('float32')
        distances, indices = self.vector_store.search(query_vector, top_k)

        results = []
        for i, idx in enumerate(indices[0]):
            if idx != -1:
                results.append((idx, float(distances[0][i])))
        return results

    def keyword_search(self, query: str, top_k: int = Config.RETRIEVAL_TOP_K) ->
List[Tuple[int, float]]:
        """关键词检索"""
        if self.bm25_index is None:
            return []

        tokenized_query = query.split()
        scores = self.bm25_index.get_scores(tokenized_query)

        top_indices = np.argsort(scores)[-top_k:][::-1]
        results = [(idx, float(scores[idx])) for idx in top_indices if
scores[idx] > 0]
        return results
```

```
    def hybrid_search(self, query: str, semantic_weight: float =
Config.SEMANTIC_WEIGHT,
                      keyword_weight: float = Config.KEYWORD_WEIGHT,
                      top_k: int = Config.RETRIEVAL_TOP_K) -> List[Tuple[int,
float, Dict]]:
        """混合检索"""
        print(f"\n🔍 开始检索: {query}")

        semantic_results = self.semantic_search(query, top_k)
        keyword_results = self.keyword_search(query, top_k)

        print(f"   语义检索找到 {len(semantic_results)} 个结果")
        print(f"   关键词检索找到 {len(keyword_results)} 个结果")

        semantic_scores = {idx: score for idx, score in semantic_results}
        keyword_scores = {idx: score for idx, score in keyword_results}

        if semantic_scores:
            max_semantic = max(semantic_scores.values())
            semantic_scores = {k: v / max_semantic for k, v in
semantic_scores.items()}

        if keyword_scores:
            max_keyword = max(keyword_scores.values())
            keyword_scores = {k: v / max_keyword for k, v in
keyword_scores.items()}

        all_indices = set(semantic_scores.keys()) | set(keyword_scores.keys())
        hybrid_scores = {}

        for idx in all_indices:
            sem_score = semantic_scores.get(idx, 0) * semantic_weight
            kw_score = keyword_scores.get(idx, 0) * keyword_weight
            hybrid_scores[idx] = sem_score + kw_score

        sorted_results = sorted(hybrid_scores.items(), key=lambda x: x[1],
reverse=True)[:top_k]

        results = []
        for idx, score in sorted_results:
            results.append((idx, score, {
                "semantic_score": semantic_scores.get(idx, 0),
                "keyword_score": keyword_scores.get(idx, 0),
                "content": self.documents[idx][:200]
            }))
            print(f"   文档{idx}: 综合分数={score:.3f}, 内容={self.documents[idx]
[:50]}...")

        return results

    def get_document(self, idx: int) -> str:
        return self.documents[idx] if 0 <= idx < len(self.documents) else ""


# ==================== 模型调用模块 ====================
```

```python
class ModelClient:
    """Deepseek模型客户端"""

    def __init__(self, model_type: str = "deepseek-v3"):
        self.model_type = model_type
        self.client = None
        self.model_name = Config.AVAILABLE_MODELS.get(model_type,
Config.AVAILABLE_MODELS["deepseek-v3"])
        self.use_local = False

    def init_client(self):
        """初始化客户端"""
        try:
            from openai import OpenAI

            if Config.DEEPSEEK_API_KEY and Config.DEEPSEEK_API_KEY != "your-
deepseek-api-key":
                self.client = OpenAI(
                    api_key=Config.DEEPSEEK_API_KEY,
                    base_url=Config.DEEPSEEK_BASE_URL
                )
                print(f"✅ Deepseek API连接成功，模型: {self.model_name}")
            else:
                self.client = OpenAI(
                    api_key="ollama",
                    base_url=Config.OLLAMA_URL
                )
                self.use_local = True
                print(f"✅ 本地Ollama连接成功，模型: {self.model_name}")
            return True
        except Exception as e:
            print(f"❌ 模型客户端初始化失败: {e}")
            return False

    def generate(self, prompt: str, system_prompt: str = "你是一个专业的AI助手。",
                 max_tokens: int = 1024, temperature: float = 0.7) -> str:
        """生成回答"""
        if self.client is None:
            if not self.init_client():
                return "模型连接失败"

        try:
            messages = [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": prompt}
            ]

            print(f"🤖 调用模型生成中...")
            response = self.client.chat.completions.create(
                model=self.model_name,
                messages=messages,
                max_tokens=max_tokens,
                temperature=temperature
            )

            return response.choices[0].message.content
```

```
        except Exception as e:
            return f"生成失败: {e}"

    def generate_with_context(self, query: str, context: str, system_prompt: str
= None) -> str:
        """基于上下文生成回答"""
        if system_prompt is None:
            if self.model_type == "deepseek-r1":
                system_prompt = """你是一个专业的AI助手。请基于提供的上下文信息回答用户问
题。
如果上下文信息不足以回答问题，请说明这一点，不要编造答案。

请按以下格式回答:
【推理过程】: 分析问题，说明如何使用上下文信息
【最终答案】: 给出准确回答"""
            else:
                system_prompt = """你是一个专业的AI助手。请基于提供的上下文信息回答用户问
题。
如果上下文信息不足以回答问题，请说明这一点，不要编造答案。"""

        prompt = f"""上下文信息:
{context}

用户问题: {query}

请基于以上上下文信息回答用户问题: """

        return self.generate(prompt, system_prompt)


# ==================== RAG应用主类 ====================
class RAGApplication:
    """RAG应用主类"""

    def __init__(self, retriever_model_type: str = "deepseek-v3",
generator_model_type: str = "deepseek-r1"):
        self.retriever_model_type = retriever_model_type
        self.generator_model_type = generator_model_type
        self.data_processor = DataProcessor()
        self.retriever = VectorRetriever()
        self.model_client = ModelClient(generator_model_type)
        self.is_initialized = False
        self.performance_metrics = []

    def initialize(self, knowledge_file: str = Config.KNOWLEDGE_FILE):
        """初始化RAG系统"""
        print("\n" + "=" * 50)
        print("初始化RAG系统...")
        print(f"检索模型: {self.retriever_model_type}")
        print(f"生成模型: {self.generator_model_type}")
        print("=" * 50)

        print("\n[1/4] 加载知识库...")
        text = self.data_processor.load_knowledge_base(knowledge_file)

        print("\n[2/4] 文本分块...")
```

```
        chunks = self.data_processor.split_text(text)

        print("\n[3/4] 构建向量索引...")
        if not self.retriever.init_embeddings():
            return False
        if not self.retriever.build_index(chunks):
            return False

        print("\n[4/4] 构建关键词索引...")
        self.retriever.build_bm25_index()

        print("\n[5/5] 初始化生成模型...")
        if not self.model_client.init_client():
            return False

        self.is_initialized = True
        print("\n✅ RAG系统初始化完成！")
        return True

    def query(self, question: str, return_details: bool = False) -> Dict:
        """执行RAG查询"""
        if not self.is_initialized:
            return {"error": "系统未初始化"}

        print("\n" + "=" * 50)
        print(f"📝 用户问题: {question}")
        print("=" * 50)

        start_time = time.time()

        retrieve_start = time.time()
        search_results = self.retriever.hybrid_search(question)
        retrieve_time = time.time() - retrieve_start

        if not search_results:
            print("⚠️ 未检索到相关文档！")
            context = "未找到相关信息。"
        else:
            context = "\n\n".join([self.retriever.get_document(idx) for idx, _, _
in search_results])
            print(f"\n📚 构建上下文，共 {len(context)} 字符")

        generate_start = time.time()
        answer = self.model_client.generate_with_context(question, context)
        generate_time = time.time() - generate_start

        total_time = time.time() - start_time

        metrics = {
            "question": question,
            "retriever_model": self.retriever_model_type,
            "generator_model": self.generator_model_type,
            "retrieve_time": retrieve_time,
            "generate_time": generate_time,
            "total_time": total_time,
            "retrieved_count": len(search_results),
```

```
            "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }
        self.performance_metrics.append(metrics)

        print(f"\n⏱ 检索耗时: {retrieve_time:.2f}s, 生成耗时:
{generate_time:.2f}s")
        print(f"📝 最终答案:\n{answer}\n")

        result = {
            "answer": answer,
            "metrics": metrics
        }

        if return_details:
            result["context"] = context
            result["retrieved_docs"] = [
                {"index": idx, "score": score, "details": details}
                for idx, score, details in search_results
            ]

        return result

    def compare_without_rag(self, question: str) -> Dict:
        """对比：不使用RAG的直接回答"""
        print("\n" + "=" * 50)
        print(f"📝 无RAG测试: {question}")
        print("=" * 50)

        start_time = time.time()
        answer = self.model_client.generate(question)
        total_time = time.time() - start_time

        print(f"📝 直接回答:\n{answer}\n")

        return {
            "answer": answer,
            "metrics": {
                "total_time": total_time,
                "with_rag": False
            }
        }


# ==================== 实验评估模块 ====================
class ExperimentEvaluator:
    """实验评估与可视化"""

    def __init__(self, rag_app: RAGApplication):
        self.rag_app = rag_app
        self.results = []

    def run_tests(self):
        """运行测试"""
        print("\n" + "=" * 50)
        print("开始实验测试...")
        print("=" * 50)
```

```
        for i, test in enumerate(Config.TEST_QUESTIONS, 1):
            print(f"\n{'='*40}")
            print(f"测试 {i}/{len(Config.TEST_QUESTIONS)} [{test['type']}]")
            print(f"{'='*40}")

            rag_result = self.rag_app.query(test['question'],
return_details=False)
            alone_result = self.rag_app.compare_without_rag(test['question'])

            self.results.append({
                "type": test['type'],
                "question": test['question'],
                "rag_answer": rag_result['answer'],
                "alone_answer": alone_result['answer'],
                "rag_metrics": rag_result['metrics'],
                "alone_metrics": alone_result['metrics'],
                "expected_keywords": test.get('expected_keywords', [])
            })

    def evaluate_accuracy(self, answer: str, expected_keywords: List[str]) ->
float:
        """评估准确率"""
        if not expected_keywords:
            return 0.5

        answer_lower = answer.lower()
        matched = 0

        for kw in expected_keywords:
            kw_lower = kw.lower()
            if kw_lower in answer_lower:
                matched += 1
            # 英文匹配
            if kw == "特朗普" and ("trump" in answer_lower or "donald" in
answer_lower):
                if kw_lower not in answer_lower:
                    matched += 1
            if kw == "总统" and ("president" in answer_lower):
                if kw_lower not in answer_lower:
                    matched += 1

        matched = min(matched, len(expected_keywords))
        return matched / len(expected_keywords)

    def generate_report(self):
        """生成实验报告"""
        print("\n" + "=" * 60)
        print("实验报告")
        print("=" * 60)

        data = []
        for r in self.results:
            rag_acc = self.evaluate_accuracy(r['rag_answer'],
r['expected_keywords'])
```

```
            alone_acc = self.evaluate_accuracy(r['alone_answer'],
r['expected_keywords'])

            data.append({
                "问题类型": r['type'],
                "RAG回答时间(s)": f"{r['rag_metrics']['total_time']:.2f}",
                "单独模型时间(s)": f"{r['alone_metrics']['total_time']:.2f}",
                "RAG准确率": f"{rag_acc:.1%}",
                "单独模型准确率": f"{alone_acc:.1%}",
                "检索文档数": r['rag_metrics']['retrieved_count']
            })

        df = pd.DataFrame(data)
        print("\n📊 性能对比表:")
        print(df.to_string(index=False))

        avg_rag_time = np.mean([r['rag_metrics']['total_time'] for r in
self.results])
        avg_alone_time = np.mean([r['alone_metrics']['total_time'] for r in
self.results])

        print(f"\n📈 平均响应时间对比:")
        print(f"   RAG方案: {avg_rag_time:.2f}秒")
        print(f"   单独模型: {avg_alone_time:.2f}秒")

        return df

    def visualize(self):
        """可视化结果"""
        import matplotlib
        matplotlib.use('Agg')  # 使用非交互式后端

        fig, axes = plt.subplots(2, 2, figsize=(14, 10))

        types = [r['type'] for r in self.results]
        rag_times = [r['rag_metrics']['total_time'] for r in self.results]
        alone_times = [r['alone_metrics']['total_time'] for r in self.results]

        x = np.arange(len(types))
        width = 0.35
        ax1 = axes[0, 0]
        bars1 = ax1.bar(x - width/2, rag_times, width, label='RAG方案',
color='#2E86AB')
        bars2 = ax1.bar(x + width/2, alone_times, width, label='单独模型',
color='#A23B72')
        ax1.set_xlabel('问题类型')
        ax1.set_ylabel('响应时间 (秒)')
        ax1.set_title('响应时间对比')
        ax1.set_xticks(x)
        ax1.set_xticklabels(types)
        ax1.legend()

        ax2 = axes[0, 1]
        rag_acc = [self.evaluate_accuracy(r['rag_answer'],
r['expected_keywords']) for r in self.results]
```

```
        alone_acc = [self.evaluate_accuracy(r['alone_answer'],
r['expected_keywords']) for r in self.results]

        bars3 = ax2.bar(x - width/2, rag_acc, width, label='RAG方案',
color='#2E86AB')
        bars4 = ax2.bar(x + width/2, alone_acc, width, label='单独模型',
color='#A23B72')
        ax2.set_xlabel('问题类型')
        ax2.set_ylabel('准确率')
        ax2.set_title('回答准确率对比')
        ax2.set_xticks(x)
        ax2.set_xticklabels(types)
        ax2.set_ylim(0, 1.1)
        ax2.legend()

        ax3 = axes[1, 0]
        retrieved_counts = [r['rag_metrics']['retrieved_count'] for r in
self.results]
        ax3.bar(types, retrieved_counts, color='#F18F01')
        ax3.set_xlabel('问题类型')
        ax3.set_ylabel('检索文档数')
        ax3.set_title('检索文档数量')

        ax4 = axes[1, 1]
        retrieve_times = [r['rag_metrics']['retrieve_time'] for r in
self.results]
        generate_times = [r['rag_metrics']['generate_time'] for r in
self.results]

        ax4.bar(types, retrieve_times, label='检索时间', color='#73AB84')
        ax4.bar(types, generate_times, bottom=retrieve_times, label='生成时间',
color='#F4D58C')
        ax4.set_xlabel('问题类型')
        ax4.set_ylabel('时间 (秒)')
        ax4.set_title('RAG时间分解')
        ax4.legend()

        plt.tight_layout()
        plt.savefig('experiment_results.png', dpi=150, bbox_inches='tight')
        print("\n📊 可视化图表已保存: experiment_results.png")


# ==================== 交互式界面 ====================
class InteractiveUI:
    """交互式界面"""

    def __init__(self):
        self.rag_app = None

    def run(self):
        """运行交互界面"""
        print("""
╔═══════════════════════════════════════════════════════════════╗
║      实验二：使用Deepseek-R1和V3构建RAG应用                    ║
║      功能：混合检索 | 多模型支持 | 可视化评估                  ║
╚═══════════════════════════════════════════════════════════════╝
```

```
        """)

        print("\n请选择【检索模型】:")
        print("1. Deepseek-V3")
        print("2. Deepseek-R1")
        print("3. Qwen2.5-7B (本地)")
        print("4. Qwen2.5-0.5B (轻量)")

        retriever_choice = input("\n请输入选项 (1-4): ").strip()
        model_map = {"1": "deepseek-v3", "2": "deepseek-r1", "3": "qwen2.5", "4":
"qwen2.5-0.5b"}
        retriever_model = model_map.get(retriever_choice, "deepseek-v3")

        print("\n请选择【生成模型】:")
        print("1. Deepseek-V3")
        print("2. Deepseek-R1 (推荐，会显示推理过程)")
        print("3. Qwen2.5-7B (本地)")
        print("4. Qwen2.5-0.5B (轻量)")

        generator_choice = input("\n请输入选项 (1-4): ").strip()
        generator_model = model_map.get(generator_choice, "deepseek-r1")

        print(f"\n✅ 配置: 检索={retriever_model}, 生成={generator_model}")

        self.rag_app = RAGApplication(
            retriever_model_type=retriever_model,
            generator_model_type=generator_model
        )

        if not self.rag_app.initialize():
            print("❌ RAG系统初始化失败")
            return

        while True:
            print("\n" + "-" * 40)
            print("请选择操作:")
            print("1. 提问 (RAG模式)")
            print("2. 运行完整实验评估")
            print("3. 查看性能指标")
            print("4. 退出")

            choice = input("\n请输入选项 (1-4): ").strip()

            if choice == "1":
                self._ask_question()
            elif choice == "2":
                self._run_experiment()
            elif choice == "3":
                self._show_metrics()
            elif choice == "4":
                print("再见! ")
                break

    def _ask_question(self):
        question = input("\n请输入您的问题: ").strip()
        if not question:
```

```
            return
        self.rag_app.query(question, return_details=True)

    def _run_experiment(self):
        evaluator = ExperimentEvaluator(self.rag_app)
        evaluator.run_tests()
        evaluator.generate_report()
        evaluator.visualize()

    def _show_metrics(self):
        if not self.rag_app.performance_metrics:
            print("暂无数据")
            return
        print("\n📊 历史性能指标:")
        for i, m in enumerate(self.rag_app.performance_metrics[-10:], 1):
            print(f"{i}. [{m['timestamp'][:16]}] {m['question'][:30]}...")
            print(f"   检索:{m['retrieve_time']:.2f}s 生成:
{m['generate_time']:.2f}s 总:{m['total_time']:.2f}s")


def main():
    ui = InteractiveUI()
    ui.run()


if __name__ == "__main__":
    main()
```

2.参考文献

[1] 中华人民共和国全国人民代表大会常务委员会. 中华人民共和国劳动法[Z]. 1994-07-05.